# Section 4.5 — Time Series Databases & Spatial Indexing

## Enhanced Study Notes | 5 Files in Series (4.1–4.5)

> ***Study Directive (5 files):*** *This is file 5 of 5 in the Section 4 database series. This section is* ***math-heavy*** *(geohash bit encoding, H3 cell resolution calculations, TSDB compression ratios, retention tier sizing). Give extra weight to* ***worked numeric examples*** *(e.g., encoding latitude/longitude bit by bit into a geohash, computing H3 cell area at each resolution level) and spatial index performance proofs.*

---

## Executive Overview

Time series and spatial data represent two of the most common “specialized” workloads that generic relational databases handle poorly. A well-chosen TSDB reduces storage by 90% and improves query speed by 100x vs PostgreSQL for metrics data. A well-chosen spatial index reduces a “find nearby points” query from O(N) to O(log N + k). This section covers both with the math to prove it.

---

## 1. Time Series Database Architecture

### 1.1 Why Generic Databases Fail for Time Series

```
Workload characteristics that break general-purpose databases:

  1. APPEND-ONLY writes: 99%+ of writes are new data points (never update old
values)
     Generic DB: allocates update-in-place storage structure (B-Tree) → wasted
overhead

  2. HIGH WRITE RATE: IoT scenario: 10K devices × 10 metrics × 1/sec = 100K
writes/sec
     PostgreSQL: ~50K inserts/sec (single node) → cannot keep up

  3. TIME-RANGE QUERIES: "all CPU readings for server-X in last 1 hour"
     PostgreSQL: needs index on (host, metric, timestamp) → large index, many I/Os
```

```
  4. HIGH CARDINALITY LABELS: 1M unique {host, service, datacenter, region}
combinations
     Prometheus: cardinality × series = index size problem

  5. AGGRESSIVE COMPRESSION: adjacent values in a time series are similar (CPU:
55%, 56%, 55%)
     Generic storage: stores each value independently → no compression benefit

  6. AUTOMATIC RETENTION: data from 1 year ago is rarely queried, should be
cheaper
     Generic DB: no built-in tier management

Quantified impact of using PostgreSQL for metrics:
  100 metrics × 1000 hosts × 1 sample/sec = 100K rows/sec
  Row size: (timestamp=8B + host=10B + metric=10B + value=8B + overhead=40B) =
76B/row
  Storage: 100K rows/sec × 76B × 86,400 sec/day = 657GB/day ← unsustainable

  InfluxDB TSM engine for same data:
  Compression: delta encoding + gorilla float compression → 1.5-3B/data point
  Storage: 100K points/sec × 2B × 86,400 = 17.3GB/day ← 38x less!
  Query speed: 100x faster (time-ordered on-disk layout, no B-Tree overhead)
```

## 1.2 TSDB Storage Engine Math

```
GORILLA FLOAT COMPRESSION (Facebook, adopted by InfluxDB/Prometheus):

  Observation: consecutive float values in a time series differ only slightly.
    CPU %: 55.2, 55.4, 55.1, 55.3, 55.0 ...
    Stored as IEEE 754 doubles (64 bits each):
      55.2 = 0100000001001011001100110011001100110011001100110011 (64 bits)
      55.4 = 0100000001001011100110011001100110011001100110011010 (64 bits)
    XOR of consecutive values:
      55.2 XOR 55.4 = 0000000000000000101100110011001100110011001100110110 (64
bits)
      The leading zeros + trailing zeros are very common!

  Gorilla encoding:
    Store first value as 64 bits.
    For each subsequent value:
      1. XOR with previous value
      2. If XOR = 0 (same value): store 1 bit: '0'
      3. If XOR has same leading/trailing zeros as previous: store 2 bits +
meaningful bits
      4. Otherwise: store 2 bits + 5 bits (leading zero count) + 6 bits (length) +
meaningful bits

  Result:
    Mostly same values (CPU stable): ~1 bit per sample
    Small changes: ~8-12 bits per sample
    Large changes: 64 bits per sample (rare)
    Average in practice: ~1.5-3 bits per sample vs 64 bits = 20-40x compression
```

```
DELTA ENCODING for timestamps:
  Raw timestamps: 1704067200, 1704067260, 1704067320, 1704067380 (Unix seconds)
  Delta-1: 60, 60, 60, 60 (all 60-second intervals)
  Delta-of-delta: 0, 0, 0 (perfectly regular → 1 bit per sample after first)
  Even irregular intervals compress well (most deltas cluster around expected
interval)

Prometheus 2-hour block example (1M time series, 1 sample/15 sec):
  Samples per series per 2h: 2 × 3600 / 15 = 480 samples
  Total samples: 1M × 480 = 480M samples
  At Gorilla compression: 480M × 1.5B = 90MB per 2-hour block
  Without compression: 480M × 16B (timestamp+value) = 7.7GB per block
  Compression ratio: 85x reduction
```

## 1.3 Downsampling Math — Retention Policy Design

```
Raw data growth calculation:
  Setup: 10,000 metrics, 1 sample/10 seconds, 8 bytes/sample

  Raw ingestion: 10,000 × 8B × 6 samples/min × 60 min × 24 hours = 6.91GB/day
  Without downsampling: 6.91GB × 365 = 2.5TB/year for 1 year of raw data

Retention hierarchy design:
  Tier 1 — Raw (10s intervals): keep 7 days
    7 × 6.91GB = 48.4GB
    Use: alerting, recent debugging

  Tier 2 — 1-minute aggregates: keep 90 days
    Compression: 6 raw samples → 1 aggregate (min/max/avg/count)
    Size: 6.91GB / 6 × 90 = 103.7GB
    Use: dashboards, medium-term trend analysis

  Tier 3 — 1-hour aggregates: keep 2 years
    60 raw samples → 1 aggregate per hour
    Size: 6.91GB / 60 × 730 = 84.2GB
    Use: capacity planning, annual review

  Total storage: 48.4 + 103.7 + 84.2 = 236.3GB (vs 2.5TB raw = 10.5x savings)

InfluxDB continuous query for downsampling:
  CREATE CONTINUOUS QUERY "downsample_1m" ON mydb
  RESAMPLE EVERY 1m FOR 2m
  BEGIN
    SELECT mean(value) AS value,
           max(value) AS max_value,
           min(value) AS min_value
    INTO "retention_90d"."metrics_1m"
    FROM "retention_7d"."metrics_raw"
    GROUP BY time(1m), *  -- * means: group by all tags
  END
```

### 1.4 TSDB Comparison

| Feature | InfluxDB | Prometheus | TimescaleDB | VictoriaMetrics |
|---|---|---|---|---|
| **Storage engine** | TSM (custom) | Custom (2h blocks) | PostgreSQL + chunks | Custom (VMstorage) |
| **Query language** | Flux / InfluxQL | PromQL | SQL | MetricsQL (PromQL compat) |
| **Clustering** | Enterprise only | Federation / Thanos | Native (PG) | Native |
| **Cardinality limit** | Medium | ~10M series | High (PG indexes) | Very High |
| **Compression** | ~50x | ~85x (Gorilla) | ~20x (PG) | ~70x |
| **Retention policies** | Native | Thanos/Cortex | PG partitioning | Native |
| **Best for** | General IoT/ metrics | Kubernetes monitoring | SQL-familiar teams | High cardinality, scale |

---

## 2. Spatial Indexing — Encoding the Earth

### 2.1 Geohash — Bit-by-Bit Worked Example

```
ENCODING San Francisco: (lat=37.7749°N, lon=-122.4194°W)

Geohash interleaves longitude bits and latitude bits:
  Even bit positions: longitude (range: -180 to +180)
  Odd bit positions:  latitude  (range:  -90 to  +90)

Step 1: Encode longitude = -122.4194
  Binary search over [-180, +180]:
  Bit 0: mid = 0.      -122.4194 < 0   → bit = 0  → range = [-180, 0]
  Bit 2: mid = -90.    -122.4194 < -90 → bit = 0  → range = [-180, -90]
  Bit 4: mid = -135.   -122.4194 > -135 → bit = 1  → range = [-135, -90]
  Bit 6: mid = -112.5. -122.4194 < -112.5 → bit = 0 → range = [-135, -112.5]
  Bit 8: mid = -123.75.-122.4194 > -123.75 → bit = 1 → range = [-123.75, -112.5]
  Longitude bits: 0, 0, 1, 0, 1, ...

Step 2: Encode latitude = 37.7749
  Binary search over [-90, +90]:
  Bit 1: mid = 0.   37.7749 > 0  → bit = 1 → range = [0, 90]
  Bit 3: mid = 45.  37.7749 < 45 → bit = 0 → range = [0, 45]
```

```
  Bit 5: mid = 22.5.37.7749 > 22.5 → bit = 1 → range = [22.5, 45]
  Bit 7: mid = 33.75.37.7749 > 33.75 → bit = 1 → range = [33.75, 45]
  Bit 9: mid = 39.375.37.7749 < 39.375 → bit = 0 → range = [33.75, 39.375]
  Latitude bits: 1, 0, 1, 1, 0, ...

Step 3: Interleave (even=lon, odd=lat):
  Position: 0  1  2  3  4  5  6  7  8  9  10 ...
  Source:   lon lat lon lat lon lat lon lat lon lat
  Bits:      0   1   0   0   1   1   0   1   1   0  ...
  = 0100110110...

Step 4: Group into 5-bit chunks and Base32 encode:
  01001 10110 ...
  = 9    q    ...
  Full 8-char geohash of SF: "9q8yy9jz"

Precision table:
  Chars | Bit pairs | Lat error  | Lon error  | Cell size
  1     | 5         | ±23°       | ±23°       | ~5000km × 5000km
  4     | 20        | ±0.7°      | ±1.4°      | ~39km × 78km
  6     | 30        | ±0.044°    | ±0.044°    | ~1.2km × 0.6km
  8     | 40        | ±0.0027°   | ±0.0027°   | ~38m × 19m
  12    | 60        | ±0.000017° | ±0.000017° | ~2.6cm (sub-centimeter)

Geohash string prefix = spatial proximity:
  "9q8yy" = San Francisco neighborhood
  "9q8yy9" = specific block within that neighborhood
  All "9q8yy*" hashes are within ~1.2km of each other

  SQL nearby query:
    SELECT * FROM venues WHERE geohash LIKE '9q8yy%';
    ← With B-Tree index on geohash: O(log N) to find prefix, then sequential scan
```

## 2.2 Geohash Edge Case — The Boundary Problem

```
Critical flaw: Geographically adjacent points can have very different geohash
prefixes.

Example — Points on opposite sides of a meridian boundary:
  Point A: lat=37.77, lon=-0.0001 (just west of meridian)
    Geohash: "gcpuuz..."
  Point B: lat=37.77, lon=+0.0001 (just east of meridian)
    Geohash: "u10hbp..."

  String distance: "gcpuuz" vs "u10hbp" = completely different prefixes!
  Yet physical distance: ~22 meters apart.

  This means: prefix query for "gcpuu" will NOT find Point B.
  Must query ALL 8 neighboring cells to guarantee finding nearby points.

Fix — always query center + 8 neighbors:
  def find_nearby(lat, lon, precision=6):
    center = geohash.encode(lat, lon, precision=precision)
```

```
    neighbors = geohash.neighbors(center)  # returns 8 adjacent hashes
    all_cells = [center] + list(neighbors.values())

    # Query all 9 cells
    placeholders = ','.join(['%s'] * len(all_cells))
    return db.execute(f"""
      SELECT *, ST_Distance(location, ST_Point(%s, %s)) AS dist
      FROM venues
      WHERE geohash9 IN ({placeholders})
      ORDER BY dist
      LIMIT 20
    """, [lon, lat] + all_cells)

  This guarantees finding all points within the search radius,
  even if they hash to a neighboring cell.
```

## 2.3 Google S2 Geometry — Math and Advantages

```
S2 maps the sphere to a unit cube using a face-centered coordinate system:

Step 1: Latitude/longitude → (x, y, z) on unit sphere
  x = cos(lat) × cos(lon)
  y = cos(lat) × sin(lon)
  z = sin(lat)
  San Francisco: x = cos(37.77°) × cos(-122.42°) = 0.792 × (-0.536) = -0.425
                 y = cos(37.77°) × sin(-122.42°) = 0.792 × (-0.844) = -0.668
                 z = sin(37.77°) = 0.611

Step 2: Project (x, y, z) onto cube face
  Determine which face: max(|x|, |y|, |z|) = |y| = 0.668 → face 2 (negative y)
  Normalize to face coordinates: (u, v) = (x/|y|, z/|y|) = (-0.636, 0.914)
  Apply quadratic transformation for more uniform cell sizes:
    s = 0.5 × sqrt(1 + 3u) if u >= 0, else 0.5 - 0.5 × sqrt(1 - 3u)

Step 3: Map (s, t) → integer cell ID (64-bit integer)
  s, t each in [0,1] → multiplied by 2^30 → 30-bit integer each
  Combined: 60 bits total (+ 3 bits for face + 1 bit = 64-bit cell ID)

S2 advantages over geohash:
  1. No polar distortion: geohash cells near poles are very distorted;
     S2 uses quadratic transformation for uniform cell areas at all latitudes.

  2. Integer IDs: sorting S2 IDs ≈ spatial locality (like Z-curve ordering)
     Can use integer range queries instead of string prefix queries.

  3. Hierarchical levels: 30 levels, resolution 0 = Earth (6 cells), level 30 =
~1cm²

  4. Better circle/region approximation:
     "All cells within 1km of point P" → compact set of S2 cell IDs
     Implementation: S2RegionCoverer finds minimum cell set covering a shape

  Cell area at different levels:
```

```
  Level  | Approximate area | Use case
  0      | 85M km²          | Continent
  6      | 1.4M km²         | Large country
  10     | 92K km²          | State/province
  13     | 1.2K km²         | City
  15     | 76 km²           | Neighborhood
  20     | 74 m²            | Building
  30     | ~1 cm²           | Sub-centimeter
```

## 2.4 H3 Hexagonal Grid — Why Hexagons Win

```
WHY HEXAGONS (not squares or triangles)?

Uniform distance to neighbors:
  Square grid: center-to-edge = 1.0, center-to-corner = 1.414 (41% longer!)
               → directional bias in distance calculations

  Hexagon grid: center-to-edge = center-to-center distance
                All 6 neighbors equidistant → no directional bias

  Impact: Uber's surge pricing zones, delivery radius calculations,
          disease spread models — all benefit from uniform neighbor distances.

H3 resolution table (Uber's specification):
  Res | Avg hex area    | Avg hex edge length | Use case
  0   | 4,357,449 km²   | 1,281 km            | Continental regions
  3   | 12,393 km²      | 59.8 km             | Metropolitan area
  6   | 36.1 km²        | 3.23 km             | District/neighborhood
  8   | 0.737 km²       | 461 m               | City block / delivery zone
  10  | 0.015 km²       | 65.9 m              | Individual building
  12  | 3.23 × 10⁻⁴ km² | 9.4 m               | Parking space

H3 cell count calculation:
  Resolution 0:  122 base cells
  Each resolution: ~7.07 child cells per parent (not exactly 7, H3 uses 7)
  Resolution r:  ~122 × 7^r cells

  Res 0:  122 cells
  Res 3:  122 × 7³ = 122 × 343 = 41,846 cells (≈ major world cities)
  Res 6:  122 × 7⁶ = 122 × 117,649 = 14,353,178 cells
  Res 10: 122 × 7¹⁰ = 122 × 282,475,249 = 34,461,980,378 cells (~34 billion)
  Res 15: 122 × 7¹⁵ ≈ 569 trillion cells (sub-meter precision)

Python example:
  import h3

  # Encode a location to H3 cell
  lat, lon = 37.7749, -122.4194  # San Francisco
  cell = h3.geo_to_h3(lat, lon, resolution=8)
  # → "8828308281fffff"

  # Find all cells within k rings (distance in cells)
  nearby = h3.k_ring(cell, k=3)  # all hexagons within 3 cells of center
```

```
# → set of 1 + 6 + 12 + 18 = 37 hexagons (k=3 ring)
# Distance: 3 × 461m ≈ 1.4km radius at resolution 8

# Polyfill a polygon
polygon = [(lat1,lon1), (lat2,lon2), ...]
cells = h3.polyfill(polygon, resolution=8)
# → all H3 cells covering the polygon (for geofence)
```

## 2.5 Spatial Index Comparison

| Index | Query type | Insert speed | Space | Nearest-neighbor | Best for |
|---|---|---|---|---|---|
| **Geohash** | Prefix match | Fast | Low | Good (with 9-cell query) | Simple proximity, SQL |
| **Quadtree** | 2D range | Fast | Medium | Good | 2D point data, non-uniform |
| **R-Tree** | Arbitrary polygon | Slow | High | Excellent | PostGIS, complex shapes |
| **S2** | Cell coverage | Fast | Low | Excellent | Google Maps, spherical |
| **H3** | Hexagonal region | Fast | Low | Excellent | Aggregation, equal-area zones |

```
Choosing a spatial index — when to use each:

  Geohash:
    ✅ You're using SQL and want a simple indexed TEXT column
    ✅ Quick prototype, single-DC deployment
    ❌ Nearest-neighbor with 9-cell query is acceptable
    ❌ Complex polygon queries (use R-Tree / PostGIS)
       Global coverage near poles (distortion)

  R-Tree (PostGIS ST_*):
    ✅ Complex polygon intersection queries
    ✅ Need exact geometry operations (ST_Intersects, ST_Contains)
    ❌ Already using PostgreSQL
       Very high update rate (R-Tree rebalancing is slow)
    Example: ST_DWithin(location, ST_Point(-122.42, 37.77), 1000) → within 1km

  S2:
```

```
    ✅
    ✅ Global-scale systems (Google Maps level)
    ✅ Integer IDs (efficient database storage and range queries)
    ❌ Need to cover arbitrary shapes with compact cell sets
       Less widely supported by off-the-shelf tools
    Example: Compute cell coverage for a geofence polygon, store integer IDs

  H3:
    ✅ Aggregation over geographic regions (Uber surge zones, census)
    ✅ Equal-area hexagons (unbiased distance calculations)
    ✅ Hierarchical aggregation (coarsen resolution for zoom-out)
    ❌ Not natively supported by most databases (need UDF or pre-compute)
    Example: Count rides per H3 cell at resolution 8, aggregate to 6 for region
view
```

---

## 3. Interview Use Cases & Design Problems

### Problem 1 — Design a Metrics System for 100K Servers (TSDB Architecture)

**Problem statement:** Design a metrics collection system for 100,000 servers. Each server emits 50 metrics every 15 seconds. Metrics must be queryable with < 500ms latency for the last 24 hours and < 5 seconds for the last year.

**Solution:**

```
Scale calculation:
  Writes: 100K servers × 50 metrics × (1/15 per sec) = 333K data points/sec
  Storage: 333K × 2 bytes (Gorilla compressed) × 86,400 sec/day = 57.6GB/day raw
  With retention: ~600GB total (7d raw + 90d 1m + 2y 1h)

Architecture — VictoriaMetrics (handles this scale):

  Ingest layer:
    Prometheus scrapes each server: pull model, 15s interval
    OR: servers push via remote_write to VictoriaMetrics
    VictoriaMetrics cluster: ingest nodes with consistent hashing by metric name

  Storage:
    Raw (7 days): VictoriaMetrics hot storage (NVMe SSD)
    1-minute rollups (90 days): VictoriaMetrics cold storage (HDD)
    1-hour rollups (2 years): S3 with VictoriaMetrics remote storage

  Query path for "last 24 hours" (< 500ms):
    Query hits hot SSD storage (all recent data)
    VictoriaMetrics: vectorized scan, Gorilla decompress
    Result: < 100ms typical for 24h range

  Query path for "last year" (< 5 seconds):
    Routes to pre-computed 1-hour rollups
```

```
    1 year × 100K series × 8760 hours = 876M data points
    At Gorilla speed: ~2 seconds → meets SLA

  Cardinality management:
    100K servers × 50 metrics = 5M unique time series
    Each series needs index entry: 5M × 64 bytes = 320MB (fits in RAM)
    Watch for: high-cardinality labels like request_id or trace_id
               These explode index size → never use unique IDs as metric labels

  Alerting:
    Grafana + Alertmanager on top of VictoriaMetrics
    Alert rule example: avg(cpu_usage{job="servers"}) > 90 for 5m

  Alternative architecture comparison:
    InfluxDB Enterprise: similar performance, better InfluxQL support, expensive
    Prometheus + Thanos: federation for long-term storage, complex ops
    TimescaleDB: SQL familiarity, but lower write throughput than specialized
TSDBs
    VictoriaMetrics: best write performance, PromQL compatible, simpler clustering
```

---

## Problem 2 — Find All Restaurants Within 1km Using Different Spatial Indexes

**Problem statement:** You have 10 million restaurant locations stored in a database. Given a user's GPS coordinates, find all restaurants within 1km. Compare three approaches and explain which to implement.

**Solution:**

```
Approach A — Naive SQL (no spatial index):
  SELECT id, name, lat, lon,
    (6371000 × acos(
      cos(radians(37.77)) × cos(radians(lat)) × cos(radians(lon) -
radians(-122.42)) +
      sin(radians(37.77)) × sin(radians(lat))
    )) AS dist
  FROM restaurants
  WHERE dist < 1000
  ORDER BY dist LIMIT 50;

  Performance: Full table scan (10M rows) → compute Haversine for all → O(N)
  Time: ~3 seconds for 10M rows. Unusable.

Approach B — Geohash with 9-cell query:
  Index: CREATE INDEX idx_restaurants_geohash ON restaurants (geohash8);

  def find_nearby(lat, lon, radius_m=1000):
    # At precision=6: each cell ≈ 1.2km × 0.6km → covers 1km radius with neighbors
    precision = 6 if radius_m <= 1000 else 5
    center = geohash.encode(lat, lon, precision)
    cells = [center] + list(geohash.neighbors(center).values())  # 9 cells
```

```
    candidates = db.execute("""
      SELECT id, name, lat, lon FROM restaurants
      WHERE geohash6 = ANY(%s)
    """, [cells])

    # Exact distance filter (Haversine on small candidate set)
    return [r for r in candidates if haversine(lat, lon, r.lat, r.lon) <=
radius_m]

  Performance: 9 prefix index lookups → O(log N) each → ~1,000–10,000 candidates
  Exact filter: Haversine on ~5,000 candidates = microseconds
  Total: < 10ms. Excellent.

Approach C — PostGIS R-Tree (most accurate):
  -- Schema
  ALTER TABLE restaurants ADD COLUMN location GEOGRAPHY(Point, 4326);
  UPDATE restaurants SET location = ST_Point(lon, lat)::GEOGRAPHY;
  CREATE INDEX idx_restaurants_geo ON restaurants USING GIST(location);

  -- Query
  SELECT id, name, ST_Distance(location, ST_Point(-122.42, 37.77)::GEOGRAPHY) AS
dist
  FROM restaurants
  WHERE ST_DWithin(location, ST_Point(-122.42, 37.77)::GEOGRAPHY, 1000)
  ORDER BY dist LIMIT 50;

  Performance: GIST index → R-Tree traversal → O(log N + k candidates)
  Time: < 5ms for 10M points. Excellent accuracy (accounts for Earth's curvature).

Comparison:
  Method        | Setup       | Query time | Accuracy | Maintenance
  Naive SQL     | None        | 3 seconds  | Exact    | None
  Geohash       | Medium      | < 10ms     | ~Good    | Recalculate on move
  PostGIS GiST  | High (ext)  | < 5ms      | Exact    | Auto (PostGIS)

Recommendation:
  New project, already using PostgreSQL → PostGIS (add extension, done)
  Non-PostgreSQL DB or custom storage → Geohash (portable, fast, simple)
  Global-scale (billions of points) → S2 or H3 with custom storage
```

---

## Problem 3 — Design a Geofencing System for 10M Users

**Problem statement:** Build a system that detects when any of 10 million users enters or exits a geographic polygon (e.g., a theme park boundary). Alert must fire within 5 seconds of crossing the boundary. Expected: 1M location updates per second.

**Solution:**

```
Scale analysis:
  10M users × 1 location update/10 seconds average = 1M updates/sec peak
  Each update: {user_id, lat, lon, timestamp} = ~40 bytes
  Ingestion bandwidth: 1M × 40B = 40MB/sec

Pre-computation — H3 index the geofence:
  Fence polygon (e.g., Disney World boundary, ~47 km²)
  H3 resolution 10: ~0.015 km² per cell → ~3,133 cells cover the park

  At startup: polyfill polygon → store as Redis SET:
    fence_cells = h3.polyfill(disney_world_polygon, resolution=10)  # 3,133 cells
    redis.sadd("geofence:disney", *fence_cells)

Location update processing pipeline:
  1M updates/sec → Kafka (partition by user_id for ordering)

  Consumer (per partition):
    def process_update(user_id, lat, lon):
      new_cell = h3.geo_to_h3(lat, lon, resolution=10)  # O(1) computation
      old_cell = redis.get(f"user:last_cell:{user_id}")  # O(1) Redis lookup

      was_inside = redis.sismember("geofence:disney", old_cell)  # O(1)
      is_inside  = redis.sismember("geofence:disney", new_cell)  # O(1)

      if not was_inside and is_inside:
          trigger_alert(user_id, "ENTERED", "disney")
      elif was_inside and not is_inside:
          trigger_alert(user_id, "EXITED", "disney")

      redis.set(f"user:last_cell:{user_id}", new_cell, ex=3600)

  Per-update cost: 3 Redis operations × 0.1ms = 0.3ms per update
  Throughput: 1 Kafka consumer = ~3,000 updates/sec
  Consumers needed: 1,000,000 / 3,000 = 333 consumers
  With Kafka partitioning: 333 partitions, 333 consumers → 1M updates/sec

Precision handling at fence boundary:
  Problem: User walking along boundary may oscillate between cells → alert flood
  Solution: Debounce — only alert after 3 consecutive consistent readings:
    redis.incr(f"user:inside_count:{user_id}") if is_inside
    redis.set(f"user:inside_count:{user_id}", 0) if not is_inside
    Alert only when count == 3 (confirmed entry)

Alert latency:
  GPS update → Kafka publish: ~100ms
  Kafka consumer processing: ~1ms
  Alert delivery (push notification): ~500ms
  Total: ~600ms << 5 second SLA

Why not R-Tree (PostGIS ST_Within)?
  1M updates/sec × ST_Within call = 1M polygon containment tests/sec
  PostGIS: ~50K polygon tests/sec per core → needs 20 cores just for geometry
  H3 approach: SMEMBER is O(1) in Redis → 10M/sec on 1 Redis node
  H3 wins: 200x more throughput at 1/20 the compute cost.
```

```
PostGIS wins for: irregular polygon shapes where H3 cell coverage is imprecise.
For most geofencing: H3 precision (15m at res 10) is more than sufficient.
```